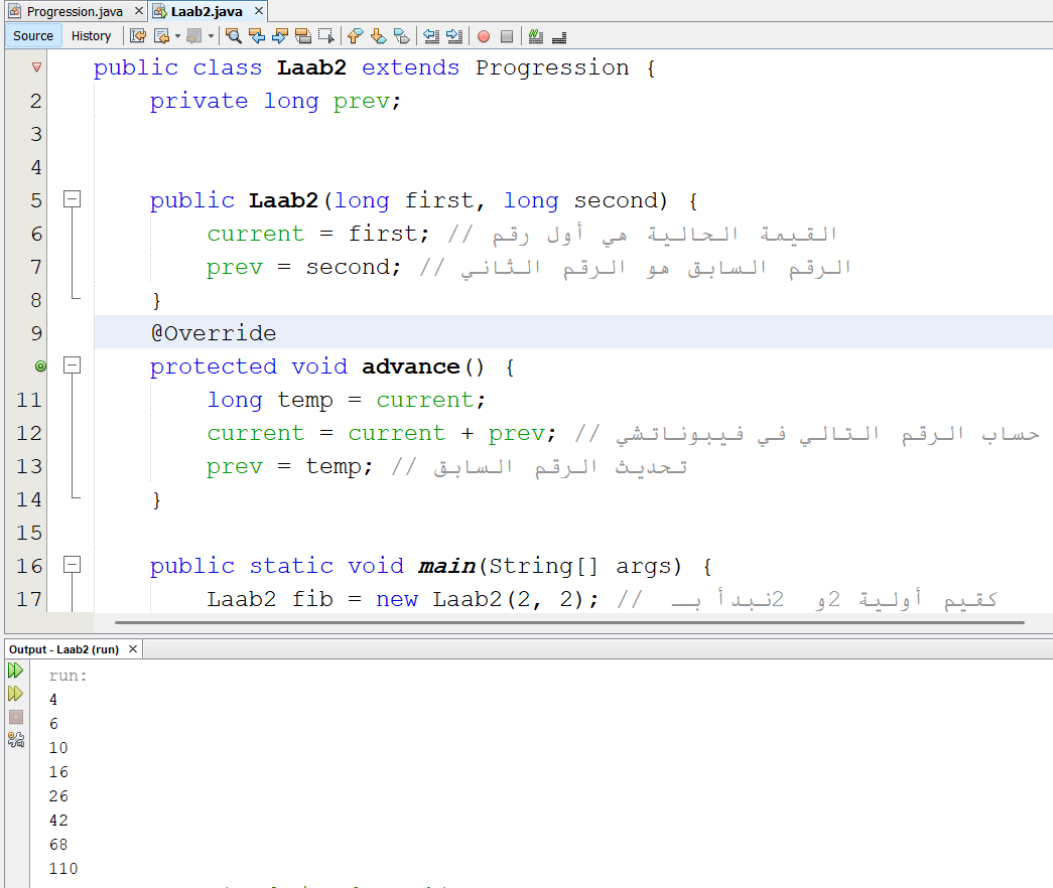


Exercises and Homework

1	R-2.4	Assume that we change the CreditCard class (see Code Fragment 1.5) so that instance variable balance has private visibility. Why is the following implementation of the PredatoryCreditCard.charge method flawed? <pre>public boolean charge(double price) { boolean isSuccess = super.charge(price); if (!isSuccess) charge(5); // the penalty return isSuccess; }</pre> لن يكون قادرًا على الوصول إلى Balance لأنه أصبح الآن private وبالتالي لن يتمكن من تعديله مباشرة داخل PredatoryCreditCard
2	R-2.5	Assume that we change the CreditCard class (see Code Fragment 1.5) so that instance variable balance has private visibility. Why is the following implementation of the PredatoryCreditCard.charge method flawed? <pre>public boolean charge(double price) { boolean isSuccess = super.charge(price); if (!isSuccess) super.charge(5); // the penalty return isSuccess; }</pre> الخلل في الكود هو أنه بعد جعل balance خاصًا (private)، لا يمكن لفئة PredatoryCreditCard الوصول إلى المتغير balance أو تعديله مباشرة. لذا، محاولة استدعاء super.charge(5) لإضافة الغرامة قد تؤدي إلى خطأ أو تصرف غير متوقع.

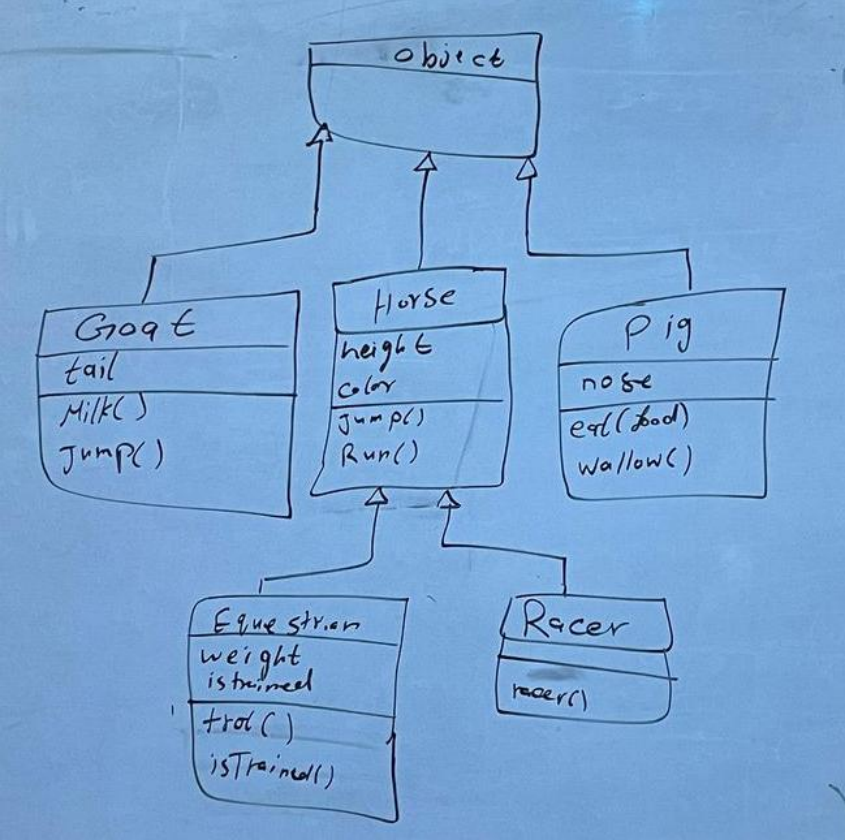
Data Structure Lab2 -Object-Oriented Design

3	R-2.6	Give a short fragment of Java code that uses the progression classes from Section 2.2.3 to find the eighth value of a Fibonacci progression that starts with 2 and 2 as its first two values.  <pre> public class Laab2 extends Progression { private long prev; public Laab2(long first, long second) { current = first; // القيمة الحالية هي أول رقم prev = second; // الرقم السابق هو الرقم الثاني } @Override protected void advance() { long temp = current; current = current + prev; // حساب الرقم التالي في فيبوناتشي prev = temp; // تحديث الرقم السابق } public static void main(String[] args) { Laab2 fib = new Laab2(2, 2); // كقيم أولية 2 و 2 نبدأ بـ } } </pre> Output - Laab2 (run) × <pre> run: 2 2 4 6 10 16 26 42 68 110 </pre>
4	R-2.7	If we choose an increment of 128, how many calls to the nextValue method from the ArithmeticProgression class of Section 2.2.3 can we make before we cause a long-integer overflow? يمكنك استدعاء طريقة nextValue () حوالي ٧٢ تريليون مرة قبل حدوث تجاوز (overflow) عند استخدام زيادة قدرها ١٢٨.
5	R-2.8	Can two interfaces mutually extend each other? Why or why not? لا، لا يمكن لواجهتين أن تمتد كل منهما من الأخرى بشكل متبادل في جافا. ذلك لأن هذا سيؤدي إلى حلقة لا نهائية من الامتداد (circular inheritance)، مما يسبب تعارضًا في التوريث ويجعل من المستحيل تحديد أي واجهة يجب أن تكون الأساس.

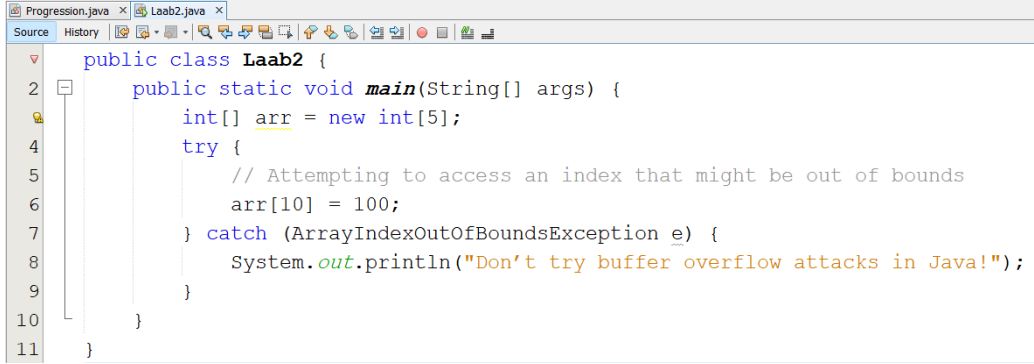
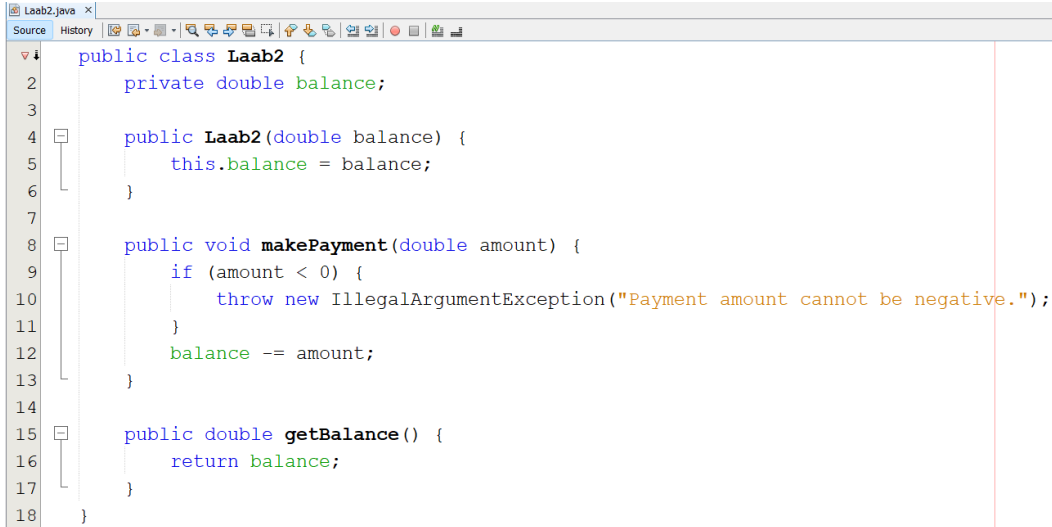
Data Structure Lab2 -Object-Oriented Design

6	R-2.9	What are some potential efficiency disadvantages of having very deep inheritance trees, that is, a large set of classes, A, B, C, and so on, such that B extends A, C extends B, D extends C, etc.? 1. تباطؤ الأداء: الوصول إلى الأعضاء في الفئات العليا قد يتطلب وقتًا أطول بسبب البحث عبر سلسلة التوريث. 2. تعقيد الصيانة: التعديلات في الفئات العليا قد تؤثر على العديد من الفئات الفرعية، مما يزيد من صعوبة الصيانة. 3. استهلاك الذاكرة: الفئات الفرعية قد تحمل بيانات وأعضاء غير مستخدمة من الفئات العليا، مما يزيد من استهلاك الذاكرة.
7	R-2.10	What are some potential efficiency disadvantages of having very shallow inheritance trees, that is, a large set of classes, A, B, C, and so on, such that all of these classes extend a single class, Z? 1. زيادة التعقيد: وجود العديد من الفئات التي تمتد من نفس الفئة قد يؤدي إلى تعقيد التصميم، مما يجعل فهم الهيكل البرمجي أصعب. 2. التوريث غير المناسب: بعض الفئات قد تحتوي على ميزات أو بيانات غير ضرورية من الفئة الأساسية، مما يؤدي إلى زيادة استهلاك الذاكرة. 3. صعوبة التوسعة: التعديلات على الفئة الأساسية قد تؤثر بشكل غير متوقع على جميع الفئات الفرعية، مما يحد من قابلية التوسعة والتعديل.
8	R-2.11	Consider the following code fragment, taken from some package: public class Maryland extends State { Maryland() { /* null constructor */ } public void printMe() { System.out.println("Read it."); } public static void main(String[] args) { Region east = new State(); State md = new Maryland(); Object obj = new Place(); Place usa = new Region(); md.printMe(); east.printMe(); ((Place) obj).printMe(); obj = md; ((Maryland) obj).printMe(); obj = usa; ((Place) obj).printMe(); usa = md; ((Place) usa).printMe(); } } class State extends Region { State() { /* null constructor */ } public void printMe() { System.out.println("Ship it."); } } class Region extends Place { Region() { /* null constructor */ } public void printMe() { System.out.println("Box it."); } } class Place extends Object { Place() { /* null constructor */ } public void printMe() { System.out.println("Buy it."); } } What is the output from calling the main() method of the Maryland class? Read it. Ship it. Buy it. Read it. Box it. Read it.

Data Structure Lab2 -Object-Oriented Design

9	R-2.1 2	Draw a class inheritance diagram for the following set of classes: • Class Goat extends Object and adds an instance variable tail and methods milk() and jump(). • Class Pig extends Object and adds an instance variable nose and methods eat(food) and wallow(). • Class Horse extends Object and adds instance variables height and color, and methods run() and jump(). • Class Racer extends Horse and adds a method race(). • Class Equestrian extends Horse and adds instance variable weight and isTrained, and methods trot() and isTrained(). 
10	R-2.1 3	Consider the inheritance of classes from Exercise R-2.12, and let d be an object variable of type Horse. If d refers to an actual object of type Equestrian, can it be cast to the class Racer? Why or why not? لا يمكن تحويل الكائن من نوع Equestrian إلى نوع Racer إذا كانت Equestrian و Racer غير مرتبطين وراثيًا. بمعنى آخر، إذا لم تكن Racer هي سلالة فرعية من Equestrian أو العكس، فإن محاولة التحويل بينهما ستؤدي إلى خطأ في وقت التشغيل (ClassCastException) في جافا. إذا كانت Equestrian و Racer غير متوارثتين عن بعضهما البعض، فإن جافا لا تسمح بالتحويل بينهما مباشرة.

Data Structure Lab2 -Object-Oriented Design

1 1	R- 2.1 4	Give an example of a Java code fragment that performs an array reference that is possibly out of bounds, and if it is out of bounds, the program catches that exception and prints the following error message: “Don’t try buffer overflow attacks in Java!”  <pre>public class Laab2 { public static void main(String[] args) { int[] arr = new int[5]; try { // Attempting to access an index that might be out of bounds arr[10] = 100; } catch (ArrayIndexOutOfBoundsException e) { System.out.println("Don't try buffer overflow attacks in Java!"); } } }</pre>
1 2	R- 2.1 5	If the parameter to the makePayment method of the CreditCard class (see Code Fragment 1.5) were a negative number, that would have the effect of raising the balance on the account. Revise the implementation so that it throws an IllegalArgumentException if a negative amount is sent as a parameter.  <pre>public class Laab2 { private double balance; public Laab2(double balance) { this.balance = balance; } public void makePayment(double amount) { if (amount < 0) { throw new IllegalArgumentException("Payment amount cannot be negative."); } balance -= amount; } public double getBalance() { return balance; } }</pre>